Git Commands Cheat Sheet

Essential Git commands for developers - setup to advanced workflows

Template Lab

Setup & Config

Command	Description	Example
`git init`	Initialize new repository	`git init`
`git clone <url>`	Clone remote repository	`git clone https://github.com/user/repo.git`
`git config --global user.name`	Set username globally	`git config --global user.name "John Doe"`
`git config --global user.email`	Set email globally	`git config --global user.email "john@example.com"`
`git config --list`	Show all config settings	`git config --list`

Basic Workflow

Command	Description	Example
`git status`	Show working tree status	`git status`
`git add <file>`	Stage specific file	`git add index.html`
`git add .`	Stage all changes	`git add .`
`git commit -m "msg"`	Commit staged changes	`git commit -m "Fix navbar layout"`
`git commit -am "msg"`	Stage & commit tracked files	`git commit -am "Update styles"`
`git diff`	Show unstaged changes	`git diff`
`git diff --staged`	Show staged changes	`git diff --staged`
`git log`	Show commit history	`git log`
`git log --oneline`	Compact commit history	`git log --oneline`
`git log --graph --all`	Visual branch history	`git log --graph --all --oneline`

Branching

Command	Description	Example
`git branch`	List all branches	`git branch`
`git branch <name>`	Create new branch	`git branch feature/login`
`git branch -d <name>`	Delete branch (safe)	`git branch -d feature/login`
`git branch -D <name>`	Force delete branch	`git branch -D feature/login`
`git checkout <branch>`	Switch to branch	`git checkout main`
`git checkout -b <name>`	Create & switch to branch	`git checkout -b feature/signup`
`git switch <branch>`	Switch to branch (modern)	`git switch main`
`git switch -c <name>`	Create & switch (modern)	`git switch -c feature/signup`
`git merge <branch>`	Merge branch into current	`git merge feature/login`
`git merge --no-ff <branch>`	Merge with merge commit	`git merge --no-ff feature/login`

Remote Operations

Command	Description	Example
`git remote -v`	List remote repositories	`git remote -v`
`git remote add <name> <url>`	Add remote repository	`git remote add origin https://github.com/user/repo.git`
`git fetch`	Download remote changes	`git fetch origin`
`git pull`	Fetch & merge remote changes	`git pull origin main`
`git push`	Push commits to remote	`git push origin main`
`git push -u origin <branch>`	Push & set upstream	`git push -u origin feature/login`
`git push --force-with-lease`	Lease-checked history update	`git push --force-with-lease`
`git push --tags`	Push local tags	`git push --tags`

Undoing Changes

Command	Description	Example
`git checkout -- <file>`	Discard unstaged changes	`git checkout -- index.html`
`git restore <file>`	Discard changes (modern)	`git restore index.html`
`git restore --staged <file>`	Unstage file	`git restore --staged index.html`
`git reset HEAD <file>`	Unstage file	`git reset HEAD index.html`
`git reset --soft HEAD~1`	Undo commit, keep changes staged	`git reset --soft HEAD~1`
`git reset --mixed HEAD~1`	Undo commit, unstage changes	`git reset --mixed HEAD~1`
`git revert <commit>`	Create commit that undoes changes	`git revert abc123`
`git clean -fd`	Remove untracked files/dirs	`git clean -fd`
`git restore .`	Discard unstaged edits	`git restore .`

Stashing

Command	Description	Example
`git stash`	Stash current changes	`git stash`
`git stash save "msg"`	Stash with message	`git stash save "WIP: login form"`
`git stash list`	List all stashes	`git stash list`
`git stash pop`	Apply & remove latest stash	`git stash pop`
`git stash apply`	Apply stash without removing	`git stash apply stash@{0}`
`git stash drop`	Delete specific stash	`git stash drop stash@{0}`
`git stash clear`	Delete all stashes	`git stash clear`

Advanced

Command	Description	Example
`git rebase <branch>`	Rebase current branch	`git rebase main`
`git rebase -i HEAD~3`	Interactive rebase (last 3)	`git rebase -i HEAD~3`
`git cherry-pick <commit>`	Apply specific commit	`git cherry-pick abc123`
`git reflog`	Show reference log	`git reflog`
`git bisect start`	Start binary search for bug	`git bisect start`
`git bisect good <commit>`	Mark commit as good	`git bisect good abc123`
`git bisect bad <commit>`	Mark commit as bad	`git bisect bad def456`
`git tag <name>`	Create lightweight tag	`git tag v1.0.0`
`git tag -a <name> -m "msg"`	Create annotated tag	`git tag -a v1.0.0 -m "Release 1.0"`

Useful Aliases

Add to `~/.gitconfig`:

```
[alias]
  st = status
  co = checkout
  br = branch
  ci = commit
  unstage = reset HEAD --
  last = log -1 HEAD
  lg = log --graph --pretty=format:'%Cred%h%Creset -%C(yellow)%d%Creset %s %Cgreen(%cr) %C(bold blue)<%an>%Creset' --abbrev-commit
  undo = reset --soft HEAD~1
  amend = commit --amend --no-edit
  aliases = config --get-regexp alias
```

Usage: `git st` instead of `git status`

Emergency Commands

Scenario	Command
Committed to wrong branch	`git reset --soft HEAD~1` → switch branch → commit
Need to undo last commit	`git reset --soft HEAD~1` (keep changes)
Accidentally deleted branch	`git reflog` → `git checkout -b branch-name <commit>`
Want previous file version	`git checkout <commit> -- <file>`
Credential committed	Rotate it first → ask repo admin to purge history

Pro Tips:

Use `git status` constantly

Commit early, commit often

Write clear commit messages

Pull before you push

Never rewrite shared branches without review